

Progetto Laboratorio Sistemi Operativi

Server Multi-Client: Battleship

Arcangelo Sansevireo

Anno Accademico 2025/2026

Indice

1	Introduzione e Architettura di Rete	2
2	Strutture Dati Principali	2
3	Gestione della Concorrenza e Sincronizzazione	3
4	Protocollo di Comunicazione	4
5	Gestione delle Disconnessioni e Robustezza	4
6	Dettagli sull'Ambiente Docker	4

1 Introduzione e Architettura di Rete

Il presente progetto implementa una versione distribuita e multi client del gioco *Battaglia Navale*. L'architettura software segue il modello **Client-Server** puro, comunicando esclusivamente tramite il protocollo TCP/IP, per garantire l'affidabilità, la corretta sequenzialità e l'integrità dei messaggi di gioco.

La parte Server è stata sviluppata in **C**, sfruttando le API POSIX per i socket e gestendo la concorrenza tramite la libreria **pthread**. La parte Client, invece, è stata sviluppata in **Python** avvalendosi del framework **Textualize**, permettendo la realizzazione di una Text-based UI (TUI) asincrona e robusta, dando una maggiore flessibilità rispetto al classico terminale.

L'ambiente di esecuzione del server è isolato e pacchettizzato utilizzando **Docker**, automatizzando il processo di build ed esecuzione attraverso un `docker-compose.yml`.

2 Strutture Dati Principali

Il design delle strutture dati è stato guidato dall'obiettivo di minimizzare le allocazioni dinamiche a runtime (`malloc/free`), riducendo così il rischio di frammentazione della memoria e leak nei thread eseguiti a lungo.

A tal proposito, la memoria necessaria per le partite viene **allocata staticamente** all'avvio all'interno di una struttura globale **GameManager**, che contiene un array fisso di dimensione `MAX_GAMES`. Ogni slot dell'array ospita una `struct Match` che funge da "sandbox" per la singola partita. Questa struttura contiene:

- **Dati identificativi e di stato:** L'ID della partita e una variabile di tipo `enum GameState` che regola l'FSM della sessione (*lobby*, *positioning*, *combat*, *game over*).
- **Modello di gioco:** Due matrici bidimensionali 10×10 che rappresentano le griglie dei giocatori, i flag di validazione per i turni e il conteggio delle navi piazzate.

Lockando lo stato di ogni partita in una struttura indipendente, si è posta la base necessaria per l'ottimizzazione della sincronizzazione tra thread.

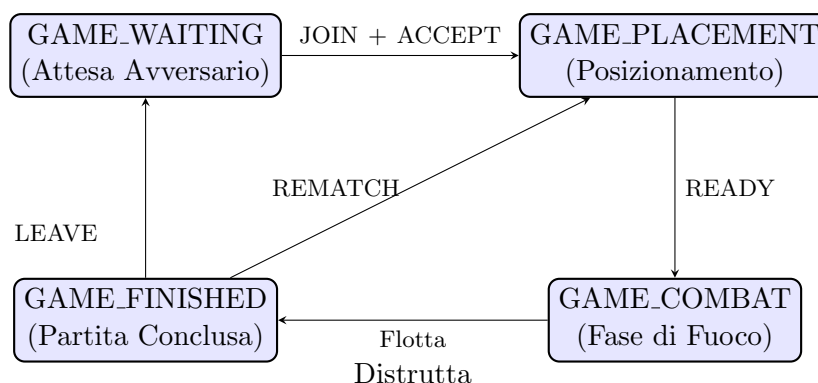


Figura 1: Automa a stati finiti (FSM) che regola il ciclo di vita di una singola istanza **Match**.

3 Gestione della Concorrenza e Sincronizzazione

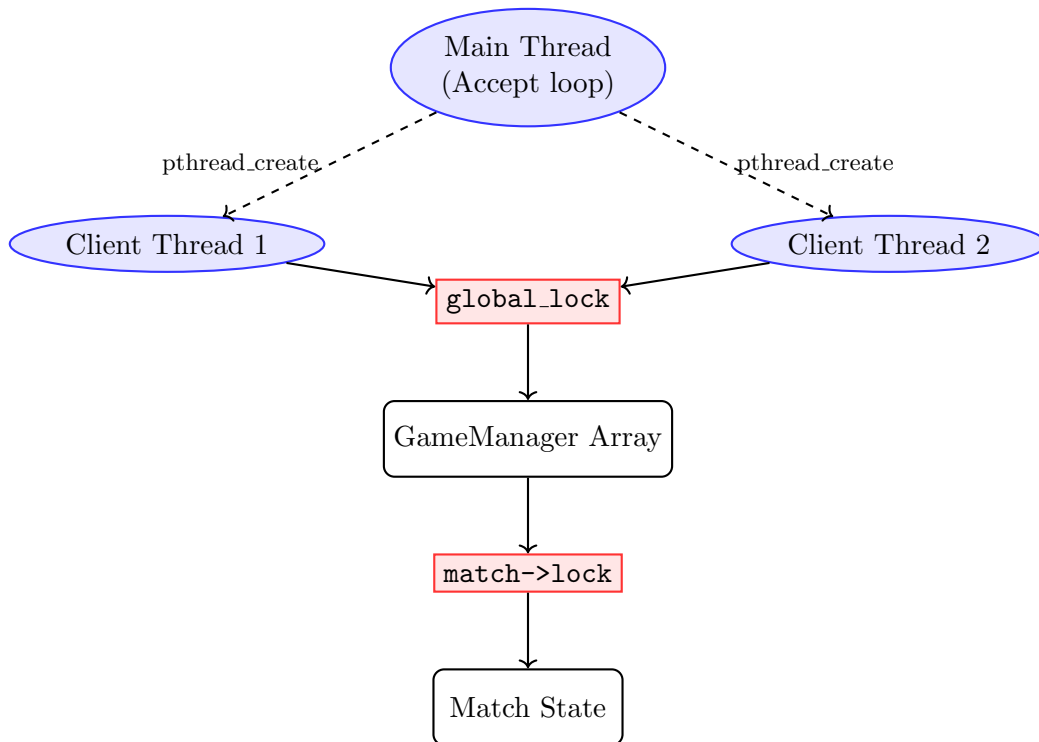


Figura 2: Gerarchia di sincronizzazione: l'accesso strutturale all'array passa per il `global_lock`, mentre le azioni di gioco richiedono solo il lock specifico della partita.

La gestione del client in **parallelo** rappresenta il core del server. Piuttosto che generare un processo figlio per ogni connessione (tramite `fork`), si è scelto di impiegare i thread POSIX (`pthread`). Questo permette a tutte le connessioni di accedere facilmente alla memoria condivisa in cui risiede il `GameManager`, ma impone una gestione della mutua esclusione per prevenire *race condition*.

Per evitare bottleneck, è stato implementato un sistema di **locking** strutturato su due livelli:

- **Global Lock** (`game_manager.global_lock`): Un mutex globale utilizzato per proteggere le operazioni strutturali sull'array delle partite, come la scansione delle partite in attesa o l'allocazione/deallocazione di una partita. L'*hold time* di questo lock è ridotto al minimo indispensabile.
- **Match Lock** (`match->lock`): Ogni singola partita ha un mutex dedicato. Quando due giocatori interagiscono nell'istanza di gioco (posizionamento e combattimento), acquisiscono solo il lock specifico della loro partita.

Quest'approccio garantisce che le operazioni di gioco di una coppia di utenti non blocchino gli altri client connessi al server. Per evitare situazioni di **deadlock**, il codice rispetta un ordine gerarchico nell'acquisizione dei lock: qualora sia necessario acquisirli entrambi, `global_lock` viene sempre bloccato prima del `match->lock`.

4 Protocollo di Comunicazione

La comunicazione tra client e server non sfrutta protocolli di alto livello preesistenti, bensì un protocollo in plain text costruito su **socket TCP**. Si è scelto un formato testuale per facilitare il parsing tramite funzioni standard della libreria `libc` (come `scanf`) e per consentire il debug manuale tramite strumenti come `netcat`.

I messaggi seguono tutti la stessa sintassi (`COMANDO [ARGOMENTO_1] [ARGOMENTO_2] ...`). Ad esempio, la fase di posizionamento usa la direttiva:

```
PLACE <match_id> <x> <y> <lunghezza> <orientamento>
```

Il server intercetta i comandi, ne valida lo stato e risponde con stringe di acknowledgment `OK` o di errore `ERR` seguite dal payload che fornisce maggiori informazioni.

5 Gestione delle Disconnessioni e Robustezza

Nel contesto della programmazione di rete, una **disconnessione anomala** del client rappresenta una delle principali cause di instabilità. Per evitare che il server subisca un arresto anomalo, sono state implementate due contromisure.

In primo luogo, il server ignora esplicitamente il segnale `SIGPIPE` all'avvio del processo principale. Di default, se il server tenta di effettuare una `write()` su un socket precedentemente chiuso dal client, il sistema invia un `SIGPIPE` che causa la terminazione del programma. Ignorando questo segnale, la funzione `write()` fallisce restituendo un codice di errore, permettendo al thread di continuare la sua esecuzione e di gestire l'anomalia.

In secondo luogo, la chiusura di una connessione, che viene rilevata dal thread quando la funzione `read()` restituisce un valore ≤ 0 , innesca obbligatoriamente una procedura di pulizia della memoria tramite la funzione `cleanup_client_matches`. Questa funzione acquisisce i mutex necessari, verifica se il client disconnesso era in una partita, notifica l'avversario (se ancora collegato) dell'abbandono e resetta lo stato della connessione. Soprattutto, questa routine assicura che i puntatori al client all'interno della struct `Match` vengano impostati a `NULL` prima che la memoria del client venga liberata (`free()`). Quest'approccio annulla il rischio di dereferenziare *dangling pointers* che causerebbero *segmentation fault*.

6 Dettagli sull'Ambiente Docker

L'ambiente di esecuzione del server è sempre isolato e configurato utilizzando **Docker-compose**. Questa scelta architetturale fornisce un ambiente di build ed esecuzione standardizzato, slegato dalle dipendenze specifiche dell'host.

Il sistema si compone di un `Dockerfile` basato sull'immagine di `gcc` che predispone gli strumenti essenziali per la compilazione tramite `Makefile`. Il file `docker-compose.yml` definisce il servizio `server`, gestendo il *port forwarding* (mappatura della porta TCP 8080 dal container dell'host) e l'aggancio dei volumi. Il montaggio della directory sorgente come volume interno al container garantisce che il codice venga compilato *from scratch* a ogni avvio del servizio mantenendo il ciclo di sviluppo rapido.